Compiling AGAMA with MS Visual Studio

Here I explain how to get the AGAMAb package (Action-based Galaxy Modelling Framework) running with Microsoft's freely-distributed 'Visual Studio' compiler. This is essential if you want to run AGAMA under Windows because it is impossible to compile Python with the Gnu compilers, and a single compiler must compile all components of a code.

Instructions for installing GSL can be found at

https://solarianprogrammer.com/2020/01/26/getting-started-gsl-gnu-scientific-library-windows-macos-linux/

In a Linux/Gnu environment, AGAMA is embodied in shared library `agamab.so`. Under VS it is contained in two files `agamab.lib` and `agamab.dll`. The former is what the linker uses, while the latter is required at runtime, so has to lie on your `path`. I have the following file `vscl.bat` on my `path` so I can compile and link a program `testit.cpp` that uses AGAMA by typing `vscl testit`

```
@echo off
cl.exe /c /openmp /I\u\c\agamab\src -EHsc /std:c++20 %1.cpp
@if not %ERRORLEVEL% == 0 (goto end)
link %1.obj agamab.lib /LIBPATH:\u\c\agamab\src\x64\release /NOIMPLIB
del %1.obj
del %1.exp
:end
```

What follows `/I` in the second line is the path to my AGAMAb source code & what follows `LIBPATH:` in the fourth line is the path to `agamab.lib`. That's where the VS IDE put it: I used the VS IDE to compile AGAMAb rather than hacking the file `makefile.local` distributed with AGAMA and using `make`. The IDE remakes faster than `make` did when I used the Gnu compilers. It manages access to the Gnu Scientific Library and Python if they were installed using the `vcpkg` utility. The last `del` command above is necessary because VS insists on producing an 'export' library file `.exp` even when told to make an executable.

Installing AGAMAb with the MSVS IDE

Clone the GitHub repository, delete `src\Py_agama.cpp` and then copy into `src` the contents of `..\windows`. Start the Visual Studio IDE and click to open a project or solution. Navigate to the src folder and click on `agamab.sln` (sln stands for 'solution'). Go to the bottom of the 'project' drop-down menu and click on 'Properties' to review choices. There's probably no need to change anything, but here's a list of things you might change:

- **Configuration Properties: General** the compiler will put the `.obj` files in the Output File, which by default is the `\x64\release` subfolder of `src`. Probably leave it like that.
- **Configuration Properties: Advanced**: 'Target File Extension' should be `.dll`
- **Linker: General** 'Output File' you might want to change the destination of the `.dll` file that is required at runtime - it must be on your `path`. By default it will go to the `x64\release` subfolder.
- **Linker: Input** 'Additional dependencies': probably leave the long list of system libraries alone

- **Linker: Advanced** 'Import library' you might want to change the destination of the `.lib` file that will be used when you link a program to AGAMAb. By default it will go to the `x64\release` subfolder.

Then provided you've installed the Gnu Scientific Library within VS, you should be able to 'build agama' from within the 'build' tab of the VS IDL. VS is a bossy compiler so there will be zillions of warnings. It will even declare an error on correct code, such one using `fopen` rather than `fopen_s`. It's easier to identify any errors if you use the drop-down menu at the top centre of the error list to change to 'Build only'.

After building AGAMA with Visual Studio Community 2019, Version 16.9.2 I tested the procedure for installing AGAMA by transferring the code to an older machine running Version 16.4.2. I encountered two problems: (i) min was said not to be a member of sdt – I fixed the problem by adding `#include <algorithm>` to `math_base.h`; (ii) the linker complained about twoPhase template instantiation. This problem was fixed by clicking Project→Properties→C/C++→Command line and typing in `/Zc:twoPhase-` at the bottom. This click sequence could also be used to establish what the compiler and linker flags should be if you prefer to hack `makefile.local` rather than using the IDE.

I've assumed that you have a 64-bit machine and that you will be running the code from the 'x64 Native Tools Command Prompt' that should be in your Visual Studio folder on your start menu. If you have a 32-bit machine, change the IDE from 'x64' to 'x86' at the top.

The following lines in a makefile

```
ASRC = \u\c\agamab\src
ALIB  = \u\c\agamab\src\x64\release\agamab.lib
ADLL  = \u\c\agamab\src\x64\release\agamab.dll
testit.exe: testit.cpp  $(ALIB) $(ADLL) other.obj
cl /openmp /I$(ASRC) -EHsc testit.cpp /link /out:testit.exe NOIMPLIB \
other.obj \libs\press.lib $(ALIB)
```

will update testit.exe when any of `testit.cpp`, `other.obj` or AGAMAb is changed. `testit` will be linked to AGAMAb, other.obj and the static library `\libs\press.lib`. The words before `/link` are instructions to the compiler, while those after are instructions to the linker.

For some reason VS doesn't put any symbols from the `.dll` it is constructing into the `.lib` file unless explicitly instructed to do so. So any symbol you want to reference from code that's not in the AGAMAb library has to have its name decorated by `__declspec(dllexport)`. This is handled by widely including `os.h` which has the line

```
#define EXP __declspec(dllexport)
```

Then `EXP` is added at appropriate places. In the case of a class, one writes

```
class EXP some_class{..
```

which will cause VS to put into the `.lib` file objects and methods defined in the class definition. When a method's code is given outside the class definition, its name needs another `EXP`:

```
EXP double some_class::some_method(int i,..){..
```

even though the method was of course declared when the class was defined. Failure to decorate the name when the code is given generates a 'redefinition with different linkage' error.

The syntax for a `struct` mirrors that of a `class`

```
struct EXP some_struct{..
```

Functions unconnected with a class are handled like methods defined outside a class

```
EXP double do_something(double x,double y){..
```

The older of my versions of Visual Studio complained when the names of symbols in internal namespaces were decorated. Since these symbols should only be referenced from within the library, this should not be a problem. Anyway don't add `EXP` within internal namespaces.

If you reference a symbol in the library that's not had its name thus decorated, it will be missing from the `.lib` file and the linker will report 'unresolved symbols'.

Implementing the Python wrappers on Windows

Access to much of the AGAMAb code from Python has been achieved via the PYBIND11 package. If you want to use our wrappers, your first step should be to install PYBIND11 by typing at a command prompt

```
pip install PYBIND11
```

Python responds to `import xx` by following the `PYTHONPATH` and there looking for a folder xx. If it finds this folder, it tries to open `__init__.py` within it. If it doesn't find the folder, it tries to open `xx.pyd` in the `PYTHONPATH`. So after installing PYBIND11, create a directory `agamab` in the `PYTHONPATH` and copy into it our file `__init__.py`, which contains

```
import os
os.add_dll_directory(''h:/u/c/agamab/src/x64/release'')
from .agamab import *
```

Edit the bit in quotes to the path to `agamab.dll` and `gsl.dll` on your machine (if they are in separate folders, use two `os.add_dll` statements). Note the period before `agamab` after `from`: this forces Python to load `agamab\agamab.pyd` rather than again opening `agamab\__init__.py`.

Within the IDE close the agamab project and open another project by navigating to the `py` folder and clicking on Py_agama.sln. Go to the bottom of the project drop-down menu and click on Properties to review choices.

- **General: Output Directory** change this to the `agamab` subfolder of the folder that PYTHONPATH points to: I've assigned this to the root of drive `k:`.
- **General: TargetName** should be `agamab`.
- **General: Configuration Type** should be Dynamic Library.
- **Advanced: Target File Extension** should be `.pyd`
- **VC++ Directories: Include Directories** must include the py_bind11 and python directories: with `k:` pointing to the PYTHONPATH folder, pybind11 is at `k:\pybind11`
- **C/C++** Additional Include Directories must cover the `agamab\src` folder where AGAMAbs header files are located
- **Linker: Additional Library Directories** must cover the location of the Python library and the location of `agamab.lib`

After the review of 'Propeties', the build tab should put agamab.pyd in the agamab subfolder at the end of PYTHONPATH. You should now be able to

```
import agamab as whatever
```

James Binney May 2021 – Nov 2025